

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is another solution to this [latex.org forum query](#).

Contents

1	Implementation	2
1.1	Package declarations	2
1.2	Overview	2
1.3	Facility layer: definitions and naming	2
1.4	Debug facilities	2
1.5	Local variables	2
1.6	Infinite loop management	2
1.7	Overlay specification	3
1.7.1	Registration	3
1.7.2	Data model	3
1.7.3	Low level IPK model functions	3
1.7.4	Loops	3
1.7.5	Path function	3
1.7.6	Low level IP model functions	3
1.7.7	Low level I model functions	3
1.7.8	Getting	3
1.7.9	Circular dependencies	3
1.7.10	Spring cleaning	3
1.7.11	The provide mode	3
1.7.12	Initialize	3
1.8	Regular expressions	4
1.9	End group setters	4
1.10	beamer.cls interface	4
1.11	Utilities	4
1.11.1	Split utilities	4
1.11.2	Match utilities	4
1.12	Main scanner	4

1.12.1 Base grammar	4
1.12.2 Storage	4
1.12.3 AST exportation policy	5
1.12.4 Branch functions	5
1.12.5 Begin and end of scanning processes	7
1.12.6 Main scan functions	7
1.12.7 $\langle Rnd \rangle$, $\langle Sqr \rangle$ and $\langle Cr1 \rangle$ units	7
1.12.8 Raw units	10
1.12.9 $\langle csl(\langle item \rangle) \rangle$ units	11
1.12.10 ISR unit	11
1.12.11 Assignment units	12
1.12.12 Call units	12
1.12.13 Resolution units	12
1.12.14 VAZL units	12

1 Implementation

1.1 Package declarations

```

1 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
2 \ProvidesExplPackage
3   {beanoves}
4   {2025/06/03}
5   {1.0}
6   {Named overlay specifications for beamer}

```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string **beanoves** or containing the case insensitive string **bnvs** delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into beamer specifications based on this data model.

The development is made easier due to a facility layer over expl.

1.3 Facility layer: definitions and naming

GOBBLED

1.4 Debug facilities

GOBBLED

1.5 Local variables

GOBBLED

1.6 Infinite loop management

GOBBLED

1.7 Overlay specification

1.7.1 Registration

GOBBLED

1.7.2 Data model

GOBBLED

1.7.3 Low level IPK model functions

GOBBLED

1.7.4 Loops

GOBBLED

1.7.5 Path function

GOBBLED

1.7.6 Low level IP model functions

1.7.7 Low level I model functions

1.7.8 Getting

GOBBLED

1.7.9 Circular dependencies

GOBBLED

1.7.10 Spring cleaning

GOBBLED

1.7.11 The provide mode

GOBBLED

1.7.12 Initialize

GOBBLED

1.8 Regular expressions

GOBBLED

```
7 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
8   \A \s* (?:
9
10      • 2 → V
11      ( [^:]+? )
12
13      • 3, 4, 5 → A : Z? or A :: L?
14      | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
15
16      • 6, 7 → ::(L:Z)?
17      | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
18
19      • 8, 9 → :(Z::L)?
20      | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
21    )
22   \s* \Z
23 }
```

GOBBLED

1.9 End group setters

GOBBLED

1.10 beamer.cls interface

GOBBLED

1.11 Utilities

1.11.1 Split utilities

GOBBLED

1.11.2 Match utilities

GOBBLED

1.12 Main scanner

GOBBLED

1.12.1 Base grammar

GOBBLED

1.12.2 Storage

GOBBLED

1.12.3 AST exportation policy

GOBBLED

1.12.4 Branch functions

The scanning process only applies to characters of category codes space and other. It definitely stops at the first token that is not of that category. It takes one of 6 branches depending on what is scanned:

- $\langle \text{Block:w} \rangle$ for unit blocks like names,
- $\langle \text{One_colon:w} \rangle$ after a single colon,
- $\langle \text{Colons:w} \rangle$ after multiple colons,
- $\langle \text{comMa:w} \rangle$ after a comma,
- $\langle \text{L:w} \rangle$ after a closing delimiter,
- $\langle \text{S:} \rangle$ when scanning is done.

Block branch The Block branch is used when the last character scanned from the $\langle \text{input stream} \rangle$ fits the current requirement as defined by the last scan function called, but the next one does not.

<code>\BNVS_scaN_Block:Fw</code>	<code>\BNVS_scaN_Block:Fw {$\langle \text{else:w} \rangle$} $\langle \text{input stream} \rangle$</code>
<code>\BNVS_scanned_B:w</code>	<code>\BNVS_scanned_B:w $\langle \text{input stream} \rangle$</code>
<code>\BNVS_scanned_set:B</code>	<code>\BNVS_scanned_set:B {$\langle \text{on_B:w} \rangle$}</code>
<code>\BNVS_scaN_Block:B:Fw</code>	<code>\BNVS_scaN_Block:B:Fw {$\langle \text{scaN:B:w} \rangle$} {$\langle \text{else:w} \rangle$} $\langle \text{input stream} \rangle$</code>
<code>\BNVS_scaN_Block_def::BF</code>	<code>\BNVS_scaN_Block_def::BF {$\langle \text{block:B:Fw} \rangle$}</code>

The `\BNVS_scanned_B:w` branch function is called by `\BNVS_scaN_Block:Fw` once a $\langle \text{block} \rangle$ has been scanned from the head of the $\langle \text{input stream} \rangle$ (greedy match). The function `\BNVS_scanned_set:B` sets the meaning of `\BNVS_scanned_B:w` to $\langle \text{Block:w} \rangle$. The exact definition of a block is driven by `\BNVS_scaN_Block:B:Fw` which meaning is set by `\BNVS_scaN_Block_def::BF`.

```
16 \cs_new:Npn \BNVS_scaN_Block:Fw #1 {
17   \BNVS_scaN_Block:B:Fw {
18     \BNVS_scanned_B:w
19   } {
20     #1
21   }
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
22 }

23 \cs_new:Npn \BNVS_scaN_Block_def::BF #1 {
24   \cs_set:Npn \BNVS_scaN_Block:B:Fw ##1 ##2 {
25     #1
26   }
27 }
```

```

28 \cs_new:Npn \BNVS_scanned_B_wrap_do:n #1 {
29   \BNVS_scanned_set:B {
30     \BNVS_scanned_B_wrapper:B { #1 }
31   }
32 }

33 \BNVS_scaN_Block_def::BF {

```

¶ At the top level, the `<else:w>` instructions are always executed.

```

34   #2
35 }

```

Colon branch **GOBBLED**

Comma branch **GOBBLED**

Close branch

`\c__bnvs_cLose_regex` Closing delimiters.

```

36 \regex_const:Nn \c__bnvs_cLose_regex { \s* (%([{
37   \}|\]|\\)
38   ) \s*
39 }

```

(End of definition for `\c__bnvs_cLose_regex`.)

<code>\BNVS_scaN_L:Fw</code>	<code>\BNVS_scaN_L:Fw {<else:w>} <input stream></code>
<code>\BNVS_scanned_L:nw</code>	<code>\BNVS_scanned_L:nw <clositer> <input stream></code>
<code>\BNVS_scanned_L:w</code>	<code>\BNVS_scanned_L:w <input stream></code>
<code>\BNVS_scanned_L_set:</code>	<code>\BNVS_scanned_set:L <on_L:w></code>

When the `\BNVS_scaN_L:Fw` function scans one of `'>`, `']>` or `'>` closing delimiter, it calls the branch function `\BNVS_scanned_L:nw` with that delimiter as argument, which in turn calls `\BNVS_scanned_L:w`, after it manages eventual errors in balancing delimiters. Otherwise execute `<else:w>` code.

```

40 \cs_new:Npn \BNVS_scaN_L:Fw #1 {
41   \peek_regex_replace_once:NnTF \c__bnvs_cLose_regex { \cB\{ \1 \cE\} } {
42     \BNVS_scanned_L:nw
43   } {
44     #1
45   }
46 }

47 \cs_new:Npn \BNVS_scanned_L:nw #1 {
48   \BNVS_error:n { Unexpected-delimiter~`#1' }
49   \BNVS_scanned_L:w
50 }

51 \tl_map_inline:nn {{Block}{One_colon}{Colons}{comMa}{cLose}} {
52   \cs_new:cpn { BNVS_scanned_#1_set:n } ##1 {
53     \cs_set:cpn { BNVS_scanned_#1:w } {

```

```

54         ##1
55     }
56 }

```

Do nothing operation at the top level.

```

57 \cs_new:cpn { BNVS_scanned_#1:w } {
58 }
59 }

60 \tl_map_inline:nn {BOCML} {
61 \cs_new:cpn { BNVS_scanned_set:#1 } ##1 {
62 \cs_set:cpn { BNVS_scanned_#1:w } {
63     ##1
64 }
65 }

```

Do nothing operation at the top level.

```

66 \cs_new:cpn { BNVS_scanned_#1:w } {
67 }
68 }

```

Stop branch **GOBBLED**

Branch setter **GOBBLED**

1.12.5 Begin and end of scanning processes

GOBBLED

1.12.6 Main scan functions

GOBBLED

1.12.7 $\langle Rnd \rangle$, $\langle Sqr \rangle$ and $\langle Cr1 \rangle$ units

We assume that delimiters are paired and balanced. Nevertheless, some errors are caught and properly managed. Here is a grammar for material enclosed between delimiters

```

1   $\langle Rnd(\langle blnc \rangle) \rangle ::= \langle Rnd\_open \rangle \langle blnc \rangle \langle Rnd\_close \rangle$ 
2   $\langle Rnd\_open \rangle ::= \langle spc \rangle '(' \langle spc \rangle$ 
3   $\langle Rnd\_close \rangle ::= \langle spc \rangle ')' \langle spc \rangle$ 
4   $\langle Sqr(\langle blnc \rangle) \rangle ::= \langle Sqr\_open \rangle \langle blnc \rangle \langle Sqr\_close \rangle$ 
5   $\langle Sqr\_open \rangle ::= \langle spc \rangle '[' \langle spc \rangle$ 
6   $\langle Sqr\_close \rangle ::= \langle spc \rangle ']' \langle spc \rangle$ 
7   $\langle Cr1(\langle blnc \rangle) \rangle ::= \langle Cr1\_open \rangle \langle blnc \rangle \langle Cr1\_close \rangle$ 
8   $\langle Cr1\_open \rangle ::= \langle spc \rangle '{' \langle spc \rangle$ 
9   $\langle Cr1\_close \rangle ::= \langle spc \rangle '}' \langle spc \rangle$ 

```

with different variations of associate AST's. (missing description for $\langle Sqr \rangle$ and $\langle Cr1 \rangle$ are similar)

```

⟨Rnd(⟨blnc⟩)⟩* := ⟨pfX:w⟩ { ⟨blnc⟩* }
                  | ⟨pfX:w⟩   ⟨blnc⟩*
                  |           { ⟨blnc⟩* }
                  |           ⟨blnc⟩*

```

The exported AST also depends on the exportation policy, whether $\langle blnc \rangle$ is empty or not.

```

\BNVS_scaN_L:N:EbsTN:w \BNVS_scaN_L:N:RBSTN:w ⟨cLose⟩
\BNVS_scaN_L:N:PXbsTN:w { ⟨R:⟩ } { ⟨scaN:B:w⟩ } { ⟨scaN:S:⟩ } { ⟨T:⟩ } { ⟨scaN:w⟩ }
\BNVS_scaN_L:N:PXBSTN:w ⟨cLose⟩
{ ⟨xprt:n⟩ } { ⟨pfX:w⟩ } { ⟨scaN:B:w⟩ } { ⟨scaN:S:⟩ } { ⟨T:⟩ } { ⟨scaN:w⟩ }

```

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using the $\langle scaN:w \rangle$ instructions. Implementation detail: we begin a scanning group that will be closed on scanning the $\langle cLose \rangle$ delimiter, or a different closing delimiter, allways at the same level. The $\langle T: \rangle$ allows some fine tuning. The $\langle scaN:w \rangle$ function scans the $\langle input\ stream \rangle$, and is expected to end on a closing delimiter at the current level by sending $\backslash\text{BNVS_scanned_L:w}$, which meaning is $\langle cLose \rangle$ followed by $\backslash\text{BNVS_scanned_L:w}$. If the $\langle scaN:w \rangle$ will call itself after a block, a comma or a colon if the corresponding branch function is called. This is the normal instruction flow.

```

69 \cs_new:Npn \BNVS_scaN_L:N:RBSTN:w #1 #2 #3 #4 #5 #6 {
70   \BNVS_scan:RBOCMLSTN:w {

```

¶ One group level for the contents.

```

71     \BNVS_begin_scan:
72     \cs_set:Npn \BNVS_scanned_L:nw ##1 {

```

¶ Anyways, branch here.

```

73         \BNVS_scanned_L:w
74     }
75     #2
76 } {

```

¶ Any branch function, but the two last ones, executes the $\langle scaN:w \rangle$ instructions.

```

77     #6
78 } {
79     #6
80 } {
81     #6
82 } {
83     #6
84 } {

```

¶ Close branch. End the group, execute $\langle block: \rangle$ instructions and calls the function $\backslash\text{BNVS_scanned_B:w}$. Intermediate groups should be closed appropriately with the right definition of branch functions. The value of I is not exported to the containing group.

```

85     \BNVS_end_scaN_no_I_change:
86     #3
87 } {

```

¶ Stop branch. Execute the $\langle stop: \rangle$ instructions after the group ends.

```

88     \BNVS_error:n { Missing~`#1'. }
89     \BNVS_end_scaN_no_I_change:
90     #4
91 } {

```

¶ Execute the $\langle T:\rangle$ instructions.

```

92     #5
93 } {

```

¶ Execute the $\langle scaN:w\rangle$ instructions.

```

94     #6
95 }
96 }

```

GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED

$\backslash c_bnvs_Crl_open_regex$ Curly opening delimiter.

(End of definition for $\backslash c_bnvs_Crl_open_regex$.)

```

97 \regex_const:Nn \c__bnvs_Crl_open_regex { \s* \{ \%}
98 \s* }

```

$\backslash BNVS_scaN_Crl:RBSTN:Fw$ $\backslash BNVS_scaN_Crl:RBSTN:Fw$ $\{ \langle R:\rangle \} \{ \langle block:\rangle \} \{ \langle stop:\rangle \} \{ \langle scaN:w\rangle \} \{ \langle F:w\rangle \}$
 $\langle input\ stream\rangle$

Scan $\langle Crl\rangle$ unit with the help of $\langle scaN:w\rangle$, then execute one of $\langle block:\rangle$ or $\langle S:\rangle$ instructions. $\langle scaN:w\rangle$ must end with one of the branch functions $\backslash BNVS_scanned_L:w$ or $\backslash BNVS_scanned_S:$. Otherwise execute $\langle F:w\rangle$ instructions.

```

99 \group_begin:
100 \char_set_catcode_group_begin:N (
101 \char_set_catcode_group_end:N )
102 \char_set_catcode_other:N \{
103 \char_set_catcode_other:N \}
104 \cs_new:Npn \BNVS_scaN_Crl:RBSTN:Fw #1 #2 #3 #4 #5 #6 (
105   \BNVS_peek_regex_remove_once:NTF \c__bnvs_Crl_open_regex (
106     \BNVS_scaN_L:N:RBSTN:w \%{
107     } ( #1 ) ( #2 ) ( #3 ) ( #4 ) ( #5 )
108   ) (
109     #6
110   )
111 )
112 \group_end:

```

1.12.8 Raw units

```

5 <Raw>      ::= <Raw_open> <call> <Rnd_close>
6 <Raw_open> ::= <spc> '(' <spc>
7 <RawLst>   ::= ( <RawRnd> | <RawSqr> | <RawCrl> | <RawOtr> ) *
8 <RawRnd>   ::= <Rnd_open> <RawLst> <Rnd_close>
9 <RawSqr>   ::= <Sqr_open> <RawLst> <Sqr_close>
10 <RawCrl>  ::= <Crl_open> <RawLst> <Crl_close>
11 <RawOtr>   ::= <characters of category other except delimiters>

```

\c__bnvs_open_Raw_regex Raw opening delimiter.

```

113 \regex_const:Nn \c__bnvs_open_Raw_regex { \s* "\(" %}
114 \s* }

```

(End of definition for \c__bnvs_open_Raw_regex.)

\BNVS_scaN_Raw_LS:w	\BNVS_scaN_Raw_LS:w <input stream>
\BNVS_scaN_Raw:nw	\BNVS_scaN_Raw:nw {<input stream>} <input stream>

Scan balanced material taking only delimiters into account. It stops on an unbalanced closing delimiter or in a stop situation. Everything in between is exported as is.

```

115 \cs_new:Npn \BNVS_scaN_Raw:nw #1 {
116   \BNVS_xprt_ast:n { #1 }
117   \BNVS_scaN_Raw_LS:w
118 }

119 \group_begin:
120 \char_set_catcode_group_begin:N <
121 \char_set_catcode_group_end:N >
122 \char_set_catcode_other:N \{
123 \char_set_catcode_other:N \}
124 \cs_new:Npn \BNVS_scaN_Raw_LS:w <

```

¶ If we find an opening delimiter.

```

125   \BNVS_scaN_Rnd:RBSTN:Fw <
126   \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >% (
127   \BNVS_ast_plc_init_mpt:n < ##1 ) >
128   > <
129   \BNVS_scanned_set:L <
130   \BNVS_end_scaN_no_I_change:
131   \BNVS_scaN_Raw_LS:w
132   >
133   \BNVS_scanned_B:w
134   > <% (
135   \BNVS_error:n < Missing~`)' > % (
136   \BNVS_xprt_ast:n < ) >
137   \BNVS_scanned_S:
138   > <
139   \BNVS_xprt_ast:n < ( > %)
140   > <
141   \BNVS_scaN_Raw_LS:w
142   > <

```

```

143 \BNVS_scaN_Sqr:RBSTN:Fw <
144 \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >[%[
145 \BNVS_ast_plc_init_mpt:n < ##1 ] >
146 > <
147 \BNVS_scanned_set:L <
148 \BNVS_end_scaN_no_I_change:
149 \BNVS_scaN_Raw_LS:w
150 >
151 \BNVS_scanned_B:w
152 > <%[
153 \BNVS_error:n < Missing~`]' > %[
154 \BNVS_xprt_ast:n < ] >
155 \BNVS_scanned_S:
156 > <
157 \BNVS_xprt_ast:n < [ > %[
158 > <
159 \BNVS_scaN_Raw_LS:w
160 > <
161 \BNVS_scaN_Cr1:RBSTN:Fw <
162 \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >{%[
163 \BNVS_ast_plc_init_mpt:n < ##1 } >
164 > <
165 \BNVS_scanned_set:L <
166 \BNVS_end_scaN_no_I_change:
167 \BNVS_scaN_Raw_LS:w
168 >
169 \BNVS_scanned_B:w
170 > <{%[
171 \BNVS_error:n < Missing~`}' > {%[
172 \BNVS_xprt_ast:n < } >
173 \BNVS_scanned_S:
174 > <
175 \BNVS_xprt_ast:n < { > %}
176 > <
177 \BNVS_scaN_Raw_LS:w
178 > <

```

❗ The head of the $\langle input\ stream \rangle$ is not an opening delimiter, we look for a closing delimiter, or a stop situation or finally export one character.

```

179 \BNVS_scaN_L:Fw < \BNVS_scaN_S:F < \BNVS_scaN_Raw:nw > >
180 > > >
181 \group_end:

```

GOBBLED

1.12.9 $\langle csl(\langle item \rangle) \rangle$ units

GOBBLED

1.12.10 ISR unit

GOBBLED

```

182 \regex_const:Nn \c__bnvs_IKS_regex {

```

1: the optional frame $\langle id \rangle$ with

2: the optional exclamation mark

```
183  (? : ( \ur{c__bnvs_S_regex} )? ( ! ) )?
```

3: followed by a non empty short name.

```
184  ( \ur{c__bnvs_S_regex} )
185 }
```

GOBBLED

1.12.11 Assignment units

GOBBLED

1.12.12 Call units

GOBBLED

1.12.13 Resolution units

GOBBLED

1.12.14 VAZL units

Here is a preliminary grammar for ranges or values:

```
12  <V>      ::= <MLS>
13  <A>      ::= <MLS>
14  <Z>      ::= <MLS>
15  <L>      ::= <MLS>
16  <VAZL>   ::= <V> | <AZL> | <ALZ>
17  <AZL>    ::= <A>? ':' <Z> ( ':' <L>? )?
18  <ALZ>    ::= <A>? ':' <L> ( ':' <Z>? )?
19  <VAZLRnd> ::= <Rnd_open> <VAZLLst> <Rnd_close>
20  <VAZLLst> ::= <spc> <VAZL> ( <comma> <VAZL> )* <spc>
21  <MLS>     ::= ( <spc> | <ISR> | <ISRC> | <ISMOR> | <Rnd> | <Sqr> | <Cr1>
    ↪ | <Rslv> | <Raw> | <other> )*
```

and the associate AST :

```
<V>*      ::= \V:n { <MLS>* }
<A>*      ::= \A:n { <MLS>* }
<Z>*      ::= \Z:n { <MLS>* }
<L>*      ::= \L:n { <MLS>* }
<AZ>*     ::= \AZ:w { <MLS>* } { <MLS>* }
<AL>*     ::= \AL:w { <MLS>* } { <MLS>* }
<ZL>*     ::= \ZL:w { <MLS>* } { <MLS>* }
<LZ>*     ::= \LZ:w { <MLS>* } { <blnc>* }
<AZL>*    ::= \AZL:w { <MLS>* } { <blnc>* } { <blnc>* }
<ALZ>*    ::= \ALZ:w { <MLS>* } { <blnc>* } { <blnc>* }
<star>*   ::= \star:
```

\BNVS_skip_OCMLS:oc:w	\BNVS_skip_OCMLS:oc:w {<D error>} {<C error>}
\BNVS_skip_OCMLS:w	\BNVS_skip_OCMLS:w

Utilities. Skip range components, to recover from errors.

```

186 \cs_set:Npn \BNVS_skip_OCMLS:oc:w #1 #2 {
187   \BNVS_subscan:RTN:w { } {
188     \BNVS_scanned_set:OCMLS {
189       #1
190       \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_skip_OCMLS:oc:w { } { }
191     } {
192       #2
193       \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_skip_OCMLS:oc:w { } { }
194     } {
195       \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_scanned_M:w
196     } {
197       \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_scanned_L:w
198     } {
199       \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_scanned_S:
200     }
201   } {
202     \BNVS_scaN_OCMLS:w
203   }
204 }

205 \cs_set:Npn \BNVS_skip_OCMLS:w {
206   \BNVS_skip_OCMLS:oc:w { } { }
207 }

```

\BNVS_scaN_VAZL_MLS:w	\BNVS_scaN_VAZL_MLS:w <input stream>
-----------------------	--------------------------------------

Scan the input stream for a <VAZL> unit. The scanning process will end on the first comma, the first unbalanced closing delimiter (at the same level) or in stop situation, calling one of \BNVS_scanned_L:w, \BNVS_scanned_M:w or \BNVS_scanned_S:. Used by \BNVS_scaN_VAZL_MLS:w.

```

208 }
209 \cs_new:Npn \BNVS_scaN_VAZL_MLS:w {

```

🔗 Create a subscan session to manage the ast policy. The <pfX> will depend on the material scanned. The first component is scanned in this session, the other ones are scanned in sub sessions.

```

210   \BNVS_subscan:OCMLSN:w {

```

🔗 A single colon has been scanned, next operations depend on the emptiness of <A>.

```

211   \BNVS_if_xprted_empty:TF {

```

🔗 Scan a : <Z> or a : <Z> :: <L> range. Keep the same scanning session for <Z>.

```

212     \BNVS_ast_plc_init:XX:n { \star: } { \:Z } { { ###1 } }
213     \BNVS_scanned_set:OC {
214       \BNVS_error:n { Unexpected~`:',~replaced-by~`::'. }
215       \BNVS_scanned_C:w
216     } {
217       \BNVS_if_xprted_empty:TF {

```

🔗 Bad : ::... <L> range.

```

218         \BNVS_error:n { Unexpected~`::...',~ignored. }
219         \BNVS_ast_plc_init_mpt:X:n { \star: } { }
220         \BNVS_skip_OCMLS:oc:w { } { }
221     } {
❧ : ⟨Z⟩ range.
222         \BNVS_ast_plc_init_mpt:X:n { } { #####1 }
223         \BNVS_xprted_brace:
224         \BNVS_scanning_ZL:w
225     }
226 } {
227     \BNVS_scaN_OCMLS:w
228 } {
229     \BNVS_xprted_brace:
230     \BNVS_scanning_AZ:w
231 }
232 } {
❧ A double colon has been scanned, next operations depend on the emptiness of ⟨A⟩.
233     \BNVS_if_xprted_empty:TF {
❧ Scan a :: ⟨L⟩ or a :: ⟨L⟩:⟨Z⟩ range. Keep the same scanning session for ⟨L⟩.
234         \BNVS_ast_plc_init:XX:n { \star: } { \:L } { { #####1 } }
235         \BNVS_scanned_set:OC {
236         \BNVS_if_xprted_empty:TF {
❧ Bad :: :...⟨L⟩ range.
237         \BNVS_error:n { Unexpected~`::...',~ignored. }
238         \BNVS_ast_plc_init_mpt:X:n { \star: } { }
239         \BNVS_skip_OCMLS:oc:w { } { }
240     } {
❧ :: ⟨L⟩ range.
241         \BNVS_ast_plc_init_mpt:X:n { } { #####1 }
242         \BNVS_xprted_brace:
243         \BNVS_scanning_LZ:w
244     }
245 } {
246     \BNVS_error:n { Unexpected~`::',~replaced-by~`:`. }
247     \BNVS_scanned_O:w
248 }
249     \BNVS_scaN_OCMLS:w
250 } {
251     \BNVS_xprted_brace:
252     \BNVS_scanning_AL:w
253 }
254 } {
255     \BNVS_end_scan: \BNVS_scanned_M:w
256 } {
257     \BNVS_end_scan: \BNVS_scanned_L:w
258 } {
259     \BNVS_end_scan: \BNVS_scanned_S:
260 } {
❧ By default, we assume a ⟨V⟩.

```

```

261     \BNVS_ast_plc_init:X { \:V }
262     \BNVS_scaN_OCMLS:w
263 }
264 }

```

\BNVS_scanning_ZL:w \BNVS_scanning_ZL:w

Utility. Scanning a : <Z> or a : <Z>::<L> range, the <Z> is already exported between braces.

```

265 \cs_new:Npn \BNVS_scanning_ZL:w {

```

❗ Create a subscan process to scan the <L> component. Only the \BNVS_scanned_M:w, \BNVS_scanned_L:w or \BNVS_scanned_S: branch functions are expected, the other ones raise.

```

266   \BNVS_subscan:RTN:w { } {
267     \BNVS_will_end:n {
268       \BNVS_if_xprted_empty:TF {
269         \BNVS_did_end:n {
270           \BNVS_ast_pfx_set:n { \:Z }
271         }
272       } {
273         \BNVS_ast_plc_init_grp:n { ##1 }
274         \BNVS_did_end:n {
275           \BNVS_ast_pfx_set:n { \:ZL }
276         }
277       }
278     }
279     \BNVS_scanned_set:OC {
280       \BNVS_error:n { Unexpected~`:...',~ignored. }
281       \BNVS_end_scan:
282       \BNVS_skip_OCMLS:oc:w { } { }
283     } {
284       \BNVS_error:n { Unexpected~`:...',~ignored. }
285       \BNVS_end_scan:
286       \BNVS_skip_OCMLS:oc:w { } { }
287     }
288   } {
289     \BNVS_scaN_OCMLS:w
290   }
291 }

```

\BNVS_scanning_LZ:w \BNVS_scanning_LZ:w

Utility. Scanning a ::<L> or a ::<L>:<Z> range, the <L> is already exported between braces.

```

292 \cs_new:Npn \BNVS_scanning_LZ:w {

```

❗ Create a subscan process to scan the <Z> component. Only the \BNVS_scanned_M:w, \BNVS_scanned_L:w or \BNVS_scanned_S: branch functions are expected, the other ones raise.

```

293   \BNVS_subscan:RTN:w { } {

```

```

294 \BNVS_will_end:n {
295   \BNVS_if_xprted_empty:TF {
296     \BNVS_did_end:n {
297       \BNVS_ast_pfx_set:n { \:L }
298     }
299   } {
300     \BNVS_ast_plc_init_grp:n { ##1 }
301     \BNVS_did_end:n {
302       \BNVS_ast_pfx_set:n { \:LZ }
303     }
304   }
305 }
306 \BNVS_scanned_set:OC {
307   \BNVS_error:n { Unexpected~`:... ',~ignored. }
308   \BNVS_end_scan:
309   \BNVS_skip_OCMLS:oc:w { } { }
310 } {
311   \BNVS_error:n { Unexpected~`:... ',~ignored. }
312   \BNVS_end_scan:
313   \BNVS_skip_OCMLS:oc:w { } { }
314 }
315 } {
316   \BNVS_scaN_OCMLS:w
317 }
318 }

```

\BNVS_scanning_AZ:w **\BNVS_scanning_AZ:w**

Utility. Scanning a $\langle A \rangle : \langle Z \rangle$ or a $\langle A \rangle : \langle Z \rangle :: \langle L \rangle$ range, only the $\langle A \rangle$ is already exported, between braces.

```

319 \cs_new:Npn \BNVS_scanning_AZ:w {

```

❗ Create a subscan process to catch the emptiness of $\langle Z \rangle$.

```

320   \BNVS_subscan:RTN:w {
321     \BNVS_ast_plc_brc:
322     \BNVS_scanned_set:OCMLS {
323       \BNVS_error:n { Unexpected~`:',~replaced~by~`::'. }
324       \BNVS_scanned_C:w
325     } {
326       \BNVS_end_scan:
327       \BNVS_scanning_AZL:w
328     } {
329       \BNVS_end_scan:
330       \BNVS_ast_plc_init:X:n { \:AZ } { ####1 }
331       \BNVS_scanned_M:w
332     } {
333       \BNVS_end_scan:
334       \BNVS_ast_plc_init:X:n { \:AZ } { ####1 }
335       \BNVS_scanned_L:w
336     } {
337       \BNVS_end_scan:
338       \BNVS_ast_plc_init:X:n { \:AZ } { ####1 }
339       \BNVS_scanned_S:

```

```

340     }
341     \BNVS_scaN_OCMLS:w
342 }
343 }

```

\BNVS_scanning_AZL:w \BNVS_scanning_AZL:w

Utility. Scanning a $\langle A \rangle : \langle Z \rangle$ range or a $\langle A \rangle : \langle Z \rangle :: \langle L \rangle$ range, the $\langle A \rangle$ and $\langle Z \rangle$ are already exported between braces. Only the `\BNVS_scanned_M:w`, `\BNVS_scanned_L:w` or `\BNVS_scanned_S:` are allowed, the other ones raise.

```

344 \cs_new:Npn \BNVS_scanning_AZL:w {
  ¶ Create a subscan process to catch the emptiness of  $\langle L \rangle$ .
345   \BNVS_subscan:RTN:w {
346     \BNVS_will_end:n {
347       \BNVS_if_xprted_empty:TF {
348         \BNVS_ast_plc_none:
349         \BNVS_did_end:n {
350           \BNVS_ast_plc_init_mpt:X:n { \:AZ } { ##1 }
351         }
352       } {
353         \BNVS_ast_plc_brc:
354         \BNVS_did_end:n {
355           \BNVS_ast_plc_init_mpt:X:n { \:AZL } { ##1 }
356         }
357       }
358     }
359     \BNVS_scanned_set:OC {
360       \BNVS_error:n { Unexpected~`:...',~ignored. }
361       \BNVS_end_scan:
362       \BNVS_skip_OCMLS:w
363     } {
364       \BNVS_error:n { Unexpected~`::... ',~ignored.. }
365       \BNVS_end_scan:
366       \BNVS_skip_OCMLS:w
367     }
368     \BNVS_scaN_OCMLS:w
369   }
370 }

```

\BNVS_scanning_AL:w \BNVS_scanning_AL:w

Utility. Scanning a $\langle A \rangle :: \langle L \rangle$ or a $\langle A \rangle :: \langle L \rangle : \langle Z \rangle$ range, only the $\langle A \rangle$ is already exported, between braces.

```

371 \cs_new:Npn \BNVS_scanning_AL:w {
  ¶ Create a subscan process to catch the emptiness of  $\langle L \rangle$ .
372   \BNVS_subscan:N:w {

```

```

373 \BNVS_ast_plc_brc:
374 \BNVS_scanned_set:OCMLS {
375     \BNVS_end_scan:
376     \BNVS_scanning_ALZ:w
377 } {
378     \BNVS_error:n { Unexpected~`::',~replaced~by~`:. }
379     \BNVS_scanned_O:w
380 } {
381     \BNVS_end_scan:
382     \BNVS_ast_plc_init:X:n { \:AL } { ####1 }
383     \BNVS_scanned_M:w
384 } {
385     \BNVS_end_scan:
386     \BNVS_ast_plc_init:X:n { \:AL } { ####1 }
387     \BNVS_scanned_L:w
388 } {
389     \BNVS_end_scan:
390     \BNVS_ast_plc_init:X:n { \:AL } { ####1 }
391     \BNVS_scanned_S:
392 % \begin{KWN.test}{bnvs:Nn=\BNVS_scaN_VAZL_MLS:w{/S:/AL}, Test_VAZL:w, eringo}
393 % \Test_VAZL:w
394 % 1 :: 2
395 % \assert_coverage:nn { TEST / top / S: } A
396 % \assert_xprted:nn { \:AL { \:n { 1 } } { \:n { 2 } } } Z
397 % \end{KWN.test}
398 % \begin{macrocode}
399 }
400 \BNVS_scaN_OCMLS:w
401 }
402 }

```

\BNVS_scanning_ALZ:w \BNVS_scanning_ALZ:w

Utility. Scanning a $\langle A \rangle :: \langle L \rangle$ or $\langle A \rangle :: \langle L \rangle : \langle Z \rangle$ range, the $\langle A \rangle$ and $\langle AL \rangle$ are already exported between braces. Only the [\BNVS_scanned_M:w](#), [\BNVS_scanned_L:w](#) or [\BNVS_scanned_S:](#) are allowed, the other ones raise.

```
403 \cs_new:Npn \BNVS_scanning_ALZ:w {
```

❧ Create a subscan process to catch the emptiness of $\langle Z \rangle$.

```

404 \BNVS_subscan:N:w {
405     \BNVS_will_end:n {
406         \BNVS_if_xprted_empty:TF {
407             \BNVS_ast_plc_none:
408             \BNVS_did_end:n {
409                 \BNVS_ast_plc_init_mpt:X:n { \:AL } { ##1 }
410             }
411         } {
412             \BNVS_ast_plc_brc:
413             \BNVS_did_end:n {
414                 \BNVS_ast_plc_init_mpt:X:n { \:ALZ } { ##1 }
415             }
416         }
417     }
418     \BNVS_scanned_set:OC {

```

```

419     \BNVS_error:n { Unexpected~`:... ',~ignored. }
420     \BNVS_end_scan:
421     \BNVS_skip_OCMLS:w
422   } {
423     \BNVS_error:n { Unexpected~`:.... ',~ignored.. }
424     \BNVS_end_scan:
425     \BNVS_skip_OCMLS:w
426   }
427   \BNVS_scaN_OCMLS:w
428 }
429 }

```

\BNVS_scaN_VAZL:MLS:w **\BNVS_scaN_VAZL:MLS:w** {<on_M:w>} {<on_L:w>} {<on_S:>} <input stream>

Scan the input stream for a <VAZL> unit. The scanning process will end on the first comma, the first unbalanced closing delimiter (at the same level) or in stop situation. The **\BNVS_scaN_OCMLS:w** instructions are used in the background to scan the <input stream>, they end with one of **\BNVS_scanned_M:w**, **\BNVS_scanned_L:w** or **\BNVS_scanned_S:**.

```

430 \cs_new:Npn \BNVS_scaN_VAZL:MLS:w #1 #2 #3 {

```

❏ Create a subscan process to manage the branch parameters.

```

431   \BNVS_subscan:MLS:N:w {
432     \BNVS_end_scan: #1
433   } {
434     \BNVS_end_scan: #2
435   } {
436     \BNVS_end_scan: #3
437   } {
438     \BNVS_ast_plc_init:n { ##1 }
439     \BNVS_scaN_VAZL:MLS:w
440   }
441 }

```

\BNVS_scaN_VAZL_brc_MLS:w **\BNVS_scaN_VAZL_brc_MLS:w** <input stream>

Scan the <input stream> for a single <VAZL> entries that is exported braced. Terminates branching to one of **\BNVS_scanned_M:w**, **\BNVS_scanned_L:w** or **\BNVS_scanned_S:**.

```

442 \cs_new:Npn \BNVS_scaN_VAZL_brc_MLS:w {

```

❏

```

443   \BNVS_subscan:N:w {
444     \BNVS_ast_plc_init:n { { ##1 } }
445     \BNVS_scaN_VAZL:MLS:w
446   }
447 }

```

\BNVS_scaN_VAZL_csl_BS:w **\BNVS_scaN_VAZL_csl_BS:w** <input stream>

Scan the <input stream> for a comma separated list of <VAZL> entries. Terminates branching to **\BNVS_scanned_S:** in stop situation, **\BNVS_scanned_B:w** otherwise.

```

448 \cs_new:Npn \BNVS_scaN_VAZL_csl_BS:w {

```

```

449 \BNVS_subscan:N:w {
450     \BNVS_scanned_set:LS {
451         \BNVS_end_scan:
452         \BNVS_scanned_B:w
453     } {
454         \BNVS_end_scan:
455         \BNVS_scanned_S:
456     }

457 \BNVS_subscan:N:w {
458     \BNVS_scanned_set:M {
459         \BNVS_end_scan:
460         \BNVS_scaN_VAZL_brc_MLS:w
461     }
462     \BNVS_scaN_VAZL_brc_MLS:w
463 }
464 }
465 }

```